docs/manuales/tecnico/manual.html

Manual Técnico
Bugs vs Code

Arquitectura, UML, orientación a objetos, algoritmos, hilos, configuración, persistencia y guía de ejecución del Proyecto 2 de IPC1.

versión: 1.0 | lenguaje: Java 21 | GUI: Swing | USAC · CUNOC | Octubre 2026

Propósito y alcance

Este manual describe la implementación del videojuego de escritorio **Bugs vs Code (Zombis vs. Plantas)**, desarrollado en Java 21 con interfaces gráficas Swing. El jugador controla a los zombis e intenta superar las plantas defensoras para alcanzar la casa.

Está dirigido a quien deba compilar el código, comprender sus clases, revisar la lógica de juego y mantener los archivos de cuentas y partidas. El contenido se basa en el código fuente incluido en el ZIP entregado, su archivo `pom.xml` y los diagramas suministrados.

> **[ALCANCE]**
> El programa no utiliza base de datos relacional. Emplea archivos TXT, JSON y binarios; por eso el apartado de persistencia describe archivos, no SQL.

Requisitos y ejecución

Requisitos

Elemento	Especificación
JDK	Java Development Kit 21 (compilación y ejecución).
Interfaz	Entorno gráfico de escritorio compatible con Swing.
Proyecto	Fuentes bajo `src/main/java` y recursos bajo `src/main/resources`.
Construcción	NetBeans o compilación con JDK. Se incluye `pom.xml` para el flujo Maven, sin librerías de terceros en tiempo de ejecución.
Almacenamiento	Permiso de escritura en la carpeta personal para cuentas y partidas.

Ejecutar desde NetBeans

1. Abrir NetBeans y seleccionar **File > Open Project**.
2. Elegir la carpeta `bugsvscode` (contiene `pom.xml`).
3. Confirmar que el proyecto usa JDK 21.
4. Ejecutar `Main.java` o la acción **Run Project** cuando esté configurada.
5. Crear o seleccionar una cuenta, y en Nueva partida elegir un nivel JSON.

Construir y ejecutar el JAR

Consola

```
mvn clean package
java -jar target/bugsvscode-1.0-SNAPSHOT.jar
```

Si se proporciona un JAR compilado, ejecutar `java -jar archivo.jar` desde una terminal. El selector de niveles permite escoger un archivo JSON externo; en el repositorio se incluye un ejemplo en `src/docs/niveles/`.

> **[COMPILACIÓN]**
> Maven aparece como herramienta de construcción del proyecto, no como dependencia de ejecución del juego. El código Java usa clases del JDK estándar.

Estructura del proyecto

El código está organizado por paquetes y separa la lógica de juego de su representación gráfica. La ruta común de clases es `src/main/java/usac/cunoc/ipc1/bugsvscode/`.

```
bugsvscode/
├── pom.xml
├── src/main/java/usac/cunoc/ipc1/bugsvscode/
│   ├── Main.java
│   ├── backend/
│   │   ├── config/           AppPaths
│   │   ├── models/           Estado, entidades y fábricas
│   │   │   ├── plants/       Plant + 7 clases
│   │   │   └── zombies/      Zombie + tipos especializados
│   │   ├── persistence/      JSON, cuentas, guardado, estadísticas
│   │   └── service/          GameEngine, combates, IA
│   └── frontend/
│       ├── panels/           Login, menú, partida
│       ├── dialogs/          Crear/seleccionar cuenta, estadísticas
│       ├── components/       GardenCanvas, controles
│       └── utils/            Sprites, fuentes, generadores
├── src/main/resources/assets/     PNG y otros recursos
└── src/docs/
    ├── niveles/                Ejemplo JSON
    └── manuales/               Documentación HTML, CSS y PDF
```

Arquitectura y responsabilidades

Presentación (frontend)

MainFrame controla la navegación; **GamePanel** administra eventos y temporizadores Swing; **GardenCanvas** dibuja el estado.

Dominio (models)

Plantas, zombis, proyectiles, cerebros, podadoras, nivel y estadísticas. Las entidades guardan sus atributos y reglas específicas.

Servicios (service)

GameEngine mantiene la partida; **PlantCombat** y **ZombieCombat** coordinan ataques; **ComputerDefense** decide siembras.

Persistencia

Repositorios para cuentas, estadísticas y partidas guardadas. **LevelReader** implementa la interpretación del JSON sin bibliotecas externas.

Flujo de comunicación

```
Arquitectura
```

```
Main → MainFrame → LoginPanel / MainMenuPanel
                        ↓
                  GamePanel → GardenCanvas (dibujo)
                        ↓
                 GameEngine → PlantCombat / ZombieCombat
                        ↓              ↘ ComputerDefense
              Entidades / arreglos
                        ↓
    Repositorios TXT / JSON / archivos .dat
```

Orientación a objetos aplicada

Encapsulamiento: el estado de las clases se mantiene en atributos privados o protegidos. **Herencia:** `Plant` concentra atributos compartidos de las siete especies, y `Zombie` los de los zombis. **Polimorfismo:** las clases especializadas proporcionan atributos o métodos particulares, mientras el motor trabaja con los tipos base. **Interfaces:** `Damageable` representa la capacidad de recibir daño. **Fábricas:** `PlantFactory` y `ZombieFactory` centralizan la construcción de entidades.

Diagramas UML

Los cinco diagramas entregados con el proyecto documentan las relaciones entre las clases principales. Se presentan por módulo para que los nombres y conexiones resulten legibles.

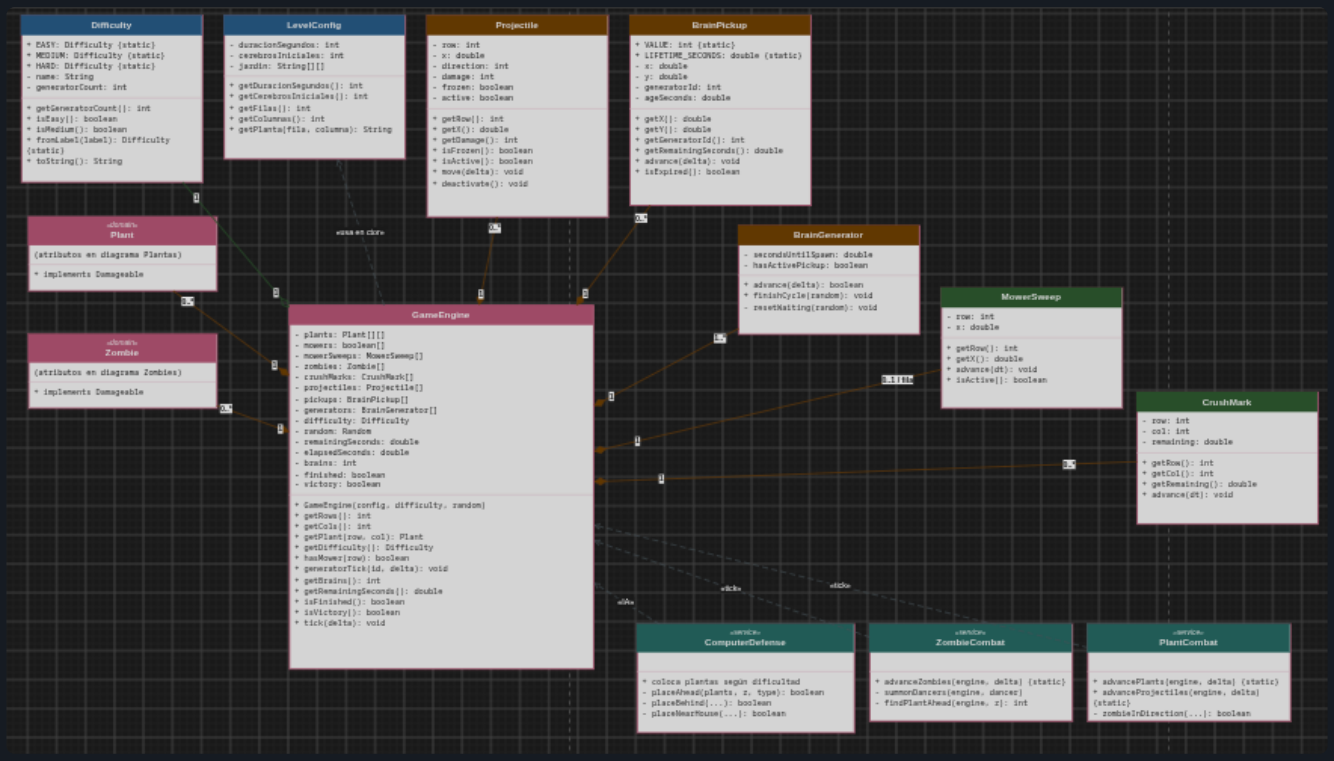


Figura 1. Motor de juego, entidades relacionadas y servicios.

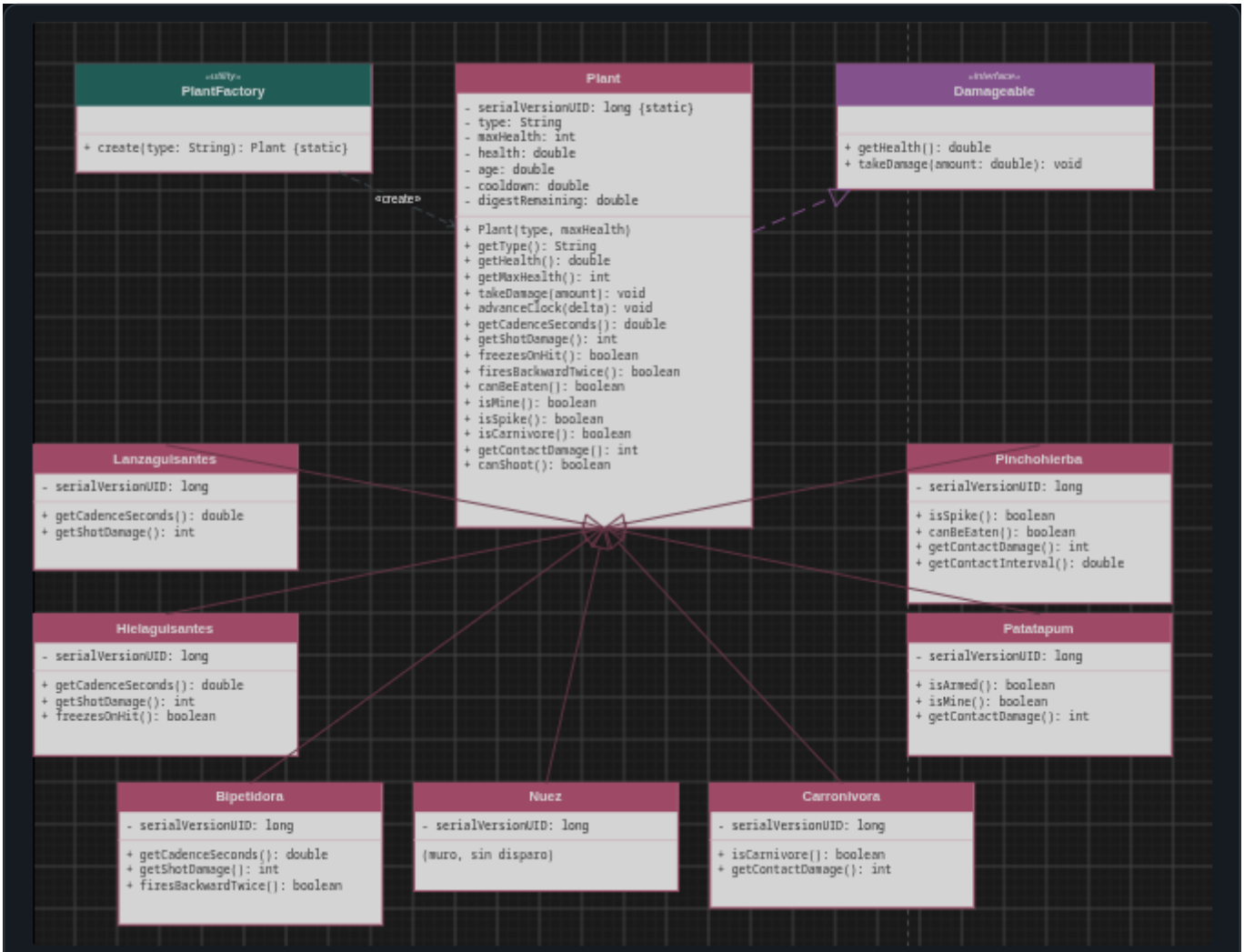


Figura 2. Clase base Plant y subclases de plantas.

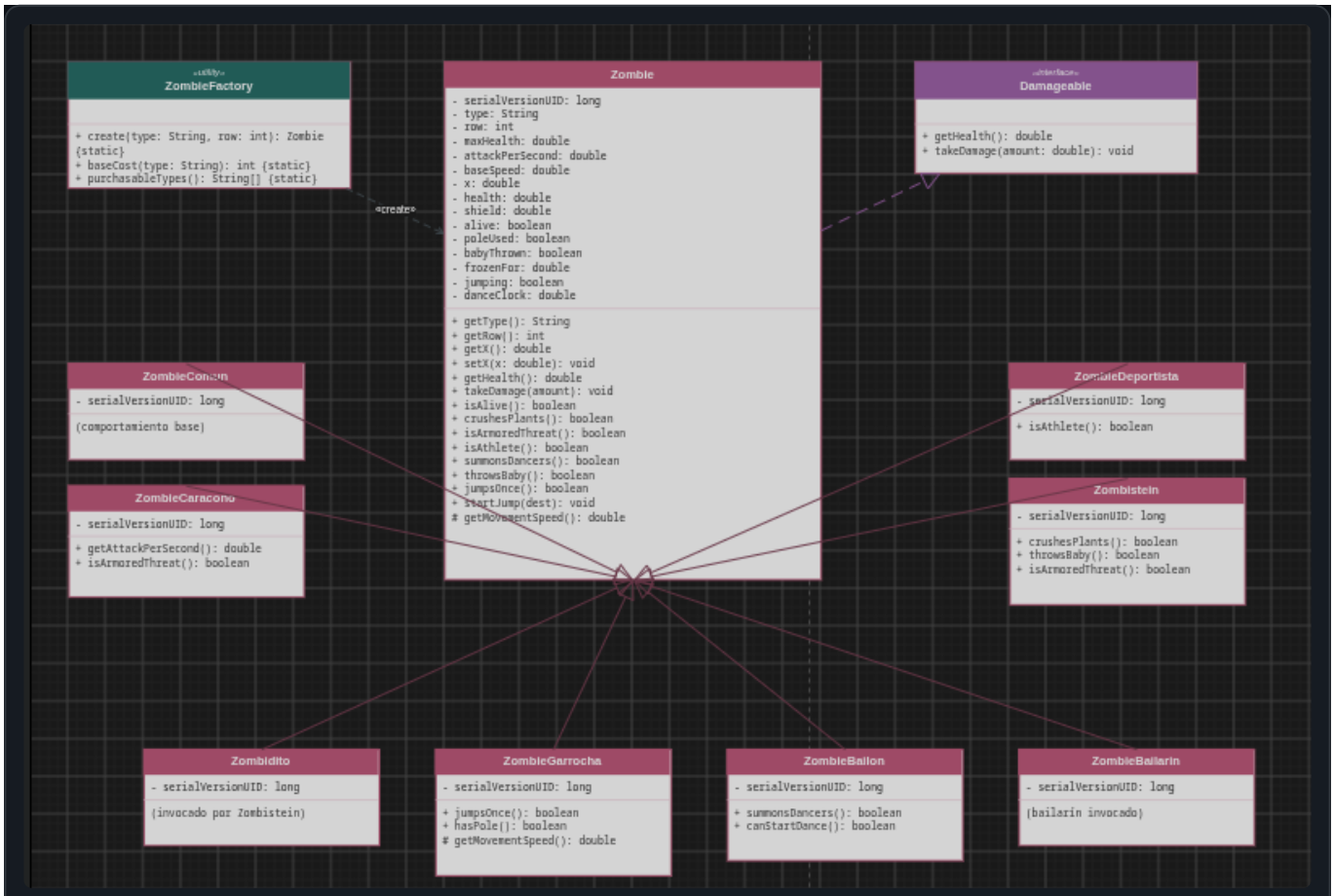


Figura 3. Clase base Zombie y tipos de zombis.

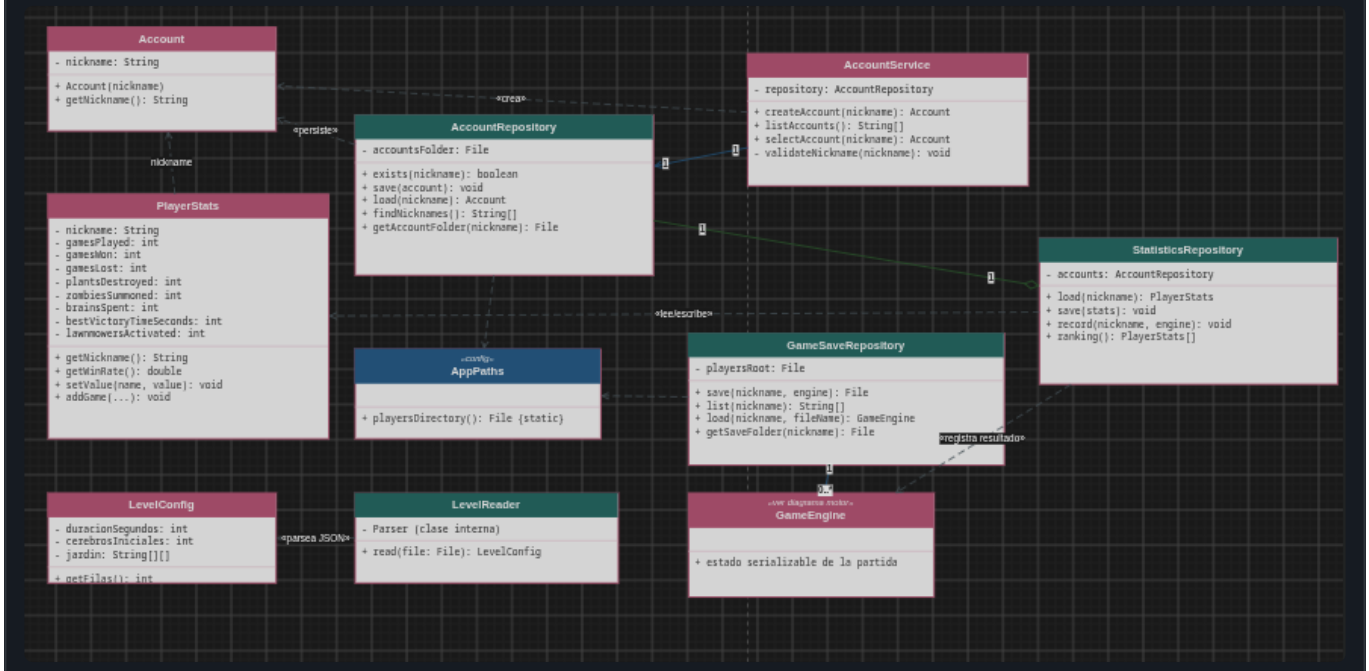


Figura 4. Repositorios y clases involucradas en persistencia.

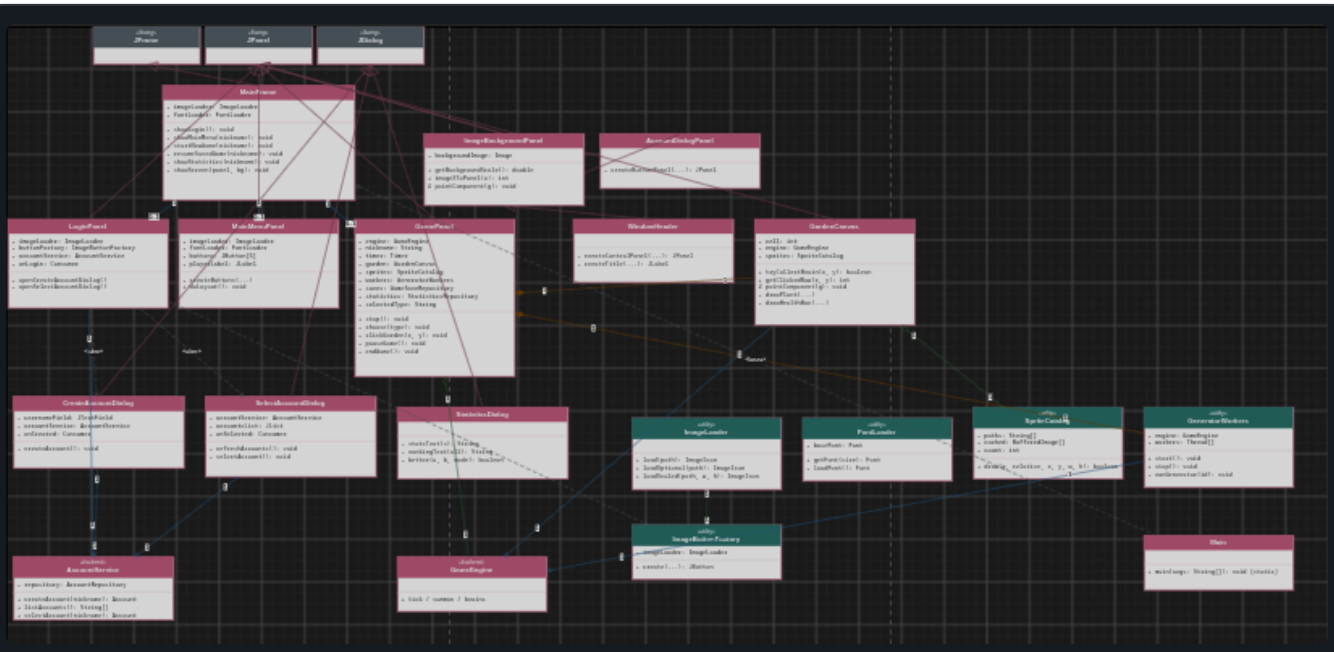


Figura 5. Jerarquía de ventanas, paneles y componentes gráficos.

Algoritmo del motor y mecánicas

Inicialización

1. El usuario selecciona un archivo y una dificultad.
2. `LevelReader` lee el JSON y devuelve `LevelConfig`.
3. `GameEngine` construye una matriz `Plant[][]` según el nivel, crea las podadoras y los generadores correspondientes.
4. `GamePanel` muestra el jardín y conecta el catálogo de zombis con los eventos del jugador.

Actualización de juego

Pseudocódigo derivado de `GameEngine.tick`

```
si la partida terminó o delta <= 0: retornar
delta = mínimo(delta, 0.10, tiempoRestante)
actualizar tiempo transcurrido y restante
si tiempoRestante <= 0: terminar como derrota
actualizar cerebros recolectables y efectos visuales
actualizar podadoras activadas
procesar ataques de plantas y proyectiles
procesar movimiento/ataques de zombis
eliminar zombis y proyectiles inactivos
si corresponde: evaluar IA de defensa
```

Colisiones y casilla lógica

El desplazamiento se representa con una posición horizontal real (`x`) y una fila (`row`). La casilla lógica se obtiene mediante `getCurrentColumn()` de cada zombi. Las interacciones verifican proximidad y carril: un proyectil afecta al primer objetivo encontrado, mientras minas, espinas, saltos e invocaciones consultan la casilla actual.

Invocación y economía

El usuario elige un tipo y hace clic en un carril. `GameEngine.invoke(type, row)` comprueba el costo, la fila y los cerebros disponibles; si procede, crea el zombi por medio de `ZombieFactory` y descuenta el costo.

La partida divide su duración total en siete bloques: en el **bloque 4** el costo se reduce 50 %, y en el **bloque 7** 75 %. Cada cerebro recolectado concede 25 unidades y permanece visible cinco segundos. Una planta destruida genera de tres a seis cerebros.

Podadoras y desenlace

Una podadora está disponible una vez por carril. Cuando un zombi alcanza la casa, elimina a los zombis del carril y activa su recorrido visual. Si un zombi llega cuando esa podadora ya fue usada, la partida finaliza con victoria; si termina el tiempo sin invasión, se registra derrota.

Defensa de la computadora

ComputerDefense recibe la matriz de plantas y la información de zombis; no dibuja componentes Swing. El motor le pide una decisión a intervalos definidos por la dificultad.

Dificultad	Frecuencia	Acción
Fácil	Sin siembra adicional	Se mantiene el jardín inicial.
Medio	Cada 10 segundos	Siembra una planta elegida al azar en una casilla disponible próxima a la casa.
Difícil	Cada 5 segundos	Evalúa condiciones en orden y realiza como máximo una siembra por evaluación.

Orden de prioridad en Difícil

1. Defender la casa ante zombis a cuatro casillas o menos, intentando colocar Nuez o Carroñívora.
2. Si un zombi supera defensas, instalar Bipetidora detrás de su posición.
3. Si hay un zombi resistente, intentar Patatapum o Carroñívora.
4. Ante deportista, priorizar Hielaguisantes; ante saltador, Nuez.
5. Ante más de tres zombis en un carril, colocar Pinchohierba.
6. Si ninguna condición puede actuar, colocar Lanzaguisantes en un carril con pocas defensas.

Pseudocódigo de ComputerDefense

```
cada 5 segundos (Difícil):
  probar prioridades en el orden establecido
  buscar casilla libre apropiada
  si una colocación se completa: finalizar evaluación
  si ninguna regla aplica: colocar defensa básica
```

> **[LIMITACIÓN CONOCIDA]**
> El código de esta versión realiza siembras directas: el formato JSON solo define cerebros del jugador y no incorpora un campo de soles para la computadora. No se describe un subsistema de soles inexistente.

Hilos, temporizadores y pausa

En la interfaz se utiliza `javax.swing.Timer` para actualizar el juego y refrescar la pantalla. La clase `GeneratorWorkers` crea tantos hilos `Thread` como indica la dificultad: 6 Fácil, 5 Medio y 4 Difícil.

Ciclo de vida de los generadores

1. **Inicio:** `start()` crea el arreglo de hilos, asigna identificadores y ejecuta `start()` sobre cada hilo.
2. **Espera:** cada hilo duerme periódicamente con `Thread.sleep(50)`.
3. **Notificación:** usa `SwingUtilities.invokeLater` para enviar al EDT el avance del generador sin modificar directamente Swing desde el hilo de trabajo.
4. **Pausa/fin:** `stop()` invalida el arreglo e interrumpe los hilos; al reanudar se vuelve a iniciar el flujo correspondiente.

Ciclo de vida

```
Hilo generador
  └── dormir 50 ms
      └── calcular delta
          └── SwingUtilities.invokeLater(...)
              └── si el generador sigue activo:
                  engine.generatorTick(id, delta)
```

La pausa detiene el reloj y la actividad visible; al guardar se serializa el estado del motor para reanudar posteriormente. Los hilos son procesos de ejecución y no componentes serializados de la partida.

Archivos, rutas y formatos

Nivel JSON

El selector usa `JFileChooser` y `LevelReader` lee un objeto con tres claves en el orden requerido: `duracionSegundos`, `cerebrosIniciales` y `jardin`. El parser propio valida cadenas reconocidas, al menos una fila y columna, y uniformidad de columnas.

JSON abreviado; formato real en src/docs/niveles/

```
{
  "duracionSegundos": 180,
  "cerebrosIniciales": 450,
  "jardin": [
    ["VACIO", "PATATAPUM", "VACIO", "NUEZ"],
    ["LANZAGUISANTES", "VACIO", "HIELAGUISANTES", "VACIO"]
  ]
}
```

Los identificadores permitidos son `VACIO`, `LANZAGUISANTES`, `PATATAPUM`, `HIELAGUISANTES`, `NUEZ`, `CARRONIVORA`, `PINCHOHIERBA` y `BIPETIDORA`.

Cuentas y estadísticas

Las rutas se centralizan en `AppPaths.playersDirectory()`, cuyo destino es `${user.home}/.bugsvscode/players/`. Cada carpeta usa `player_` seguido del nickname codificado en hexadecimal; así se conserva la distinción entre mayúsculas y minúsculas.

```
~/.bugsvscode/players/
└── player_<nickname_en_hexadecimal>/
```

```
    ├── statistics.txt
    └── saved-games/
        └── partida_<fecha_hora>.dat
```

TXT de ejemplo de formato; cifras ilustrativas

```
nickname=Panchito24
gamesPlayed=10
gamesWon=8
gamesLost=2
plantsDestroyed=56
zombiesSummoned=102
brainsSpent=9817
bestVictoryTimeSeconds=40
lawnmowersActivated=16
```

`StatisticsRepository` lee y escribe un único archivo TXT por cuenta y consolida las estadísticas al terminar una partida. `GameSaveRepository` utiliza `ObjectOutputStream` y `ObjectInputStream` para guardar/cargar el objeto `GameEngine` en `.dat`.

> **[COMPATIBILIDAD]**
> Los archivos .dat guardan objetos serializados, por lo que los cambios de estructura entre versiones pueden afectar su compatibilidad. Respaldar la carpeta de jugadores antes de migrar.

Verificación y casos de prueba

Para validar la entrega se recomienda ejecutar los siguientes casos con un nivel JSON que contenga plantas de distintos tipos. Son **casos de prueba propuestos**; esta documentación no equivale a un certificado de ausencia de defectos.

Prueba	Acción	Resultado esperado
Cuenta	Crear nickname y repetirlo	Rechazo de duplicados y creación de carpeta/TXT.
Selección	Seleccionar un usuario existente	Entrada al menú con la cuenta correcta.
JSON válido	Abrir nivel del repositorio	Jardín con las dimensiones indicadas.
JSON inválido	Carriles de distinta longitud	Mensaje de error; partida no iniciada.
Invocación	Elegir zombi y carril	Zombi aparece y se descuentan cerebros.
Recolección	Hacer clic en cerebro antes de 5 s	Aumenta en 25 cerebros.
Pausa/guardado	Pausar, guardar, salir y reanudar	Recupera estado y permanece pausada hasta continuar.
Final	Activar podadora y alcanzar casa nuevamente	Victoria y actualización de estadísticas.
Ranking	Abrir Sala de la fama	Estadísticas personales y los tres Top 5.

Errores y mantenimiento

Si el JAR no inicia, verificar `java -version` y JDK 21. Si faltan imágenes, revisar los recursos empaquetados bajo `assets/images/`. Si falla el JSON, respetar las claves y la misma cantidad de columnas por carril. Para estudiar un error del juego, empezar en la clase que maneja la responsabilidad: pantalla (`GamePanel`), reglas (`GameEngine` y combates) o archivo (repositorios).

Para mantenimiento, conservar la separación frontend/backend y documentar cambios en los constructores de entidades, la lectura de archivos y la serialización antes de distribuir una nueva versión.